

Optimization

3.2.3 Nelder–Mead simplex method

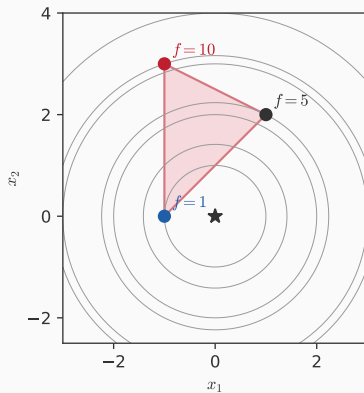
José Firmino Aguilar Madeira

From the grid to the simplex: can we use what we have already evaluated?

The grid (3.2.2) finds promising regions, but keeps spending evaluations where we already know it does not matter: as in random search, the points **do not use what has already happened**.

In 2D, with $f(x_1, x_2) = x_1^2 + x_2^2$, we evaluated three points:

$$f(-1, 0) = 1, \quad f(1, 2) = 5, \quad f(-1, 3) = 10.$$



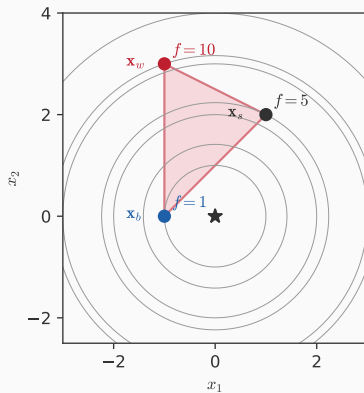
From the grid to the simplex: can we use what we have already evaluated?

The grid (3.2.2) finds promising regions, but keeps spending evaluations where we already know it does not matter: as in random search, the points **do not use what has already happened**.

In 2D, with $f(x_1, x_2) = x_1^2 + x_2^2$, we evaluated three points:

$$f(-1, 0) = 1, \quad f(1, 2) = 5, \quad f(-1, 3) = 10.$$

We sort: $f(x_b) < f(x_s) < f(x_w)$ — the **best**, the **second worst** and the **worst**.



From the grid to the simplex: can we use what we have already evaluated?

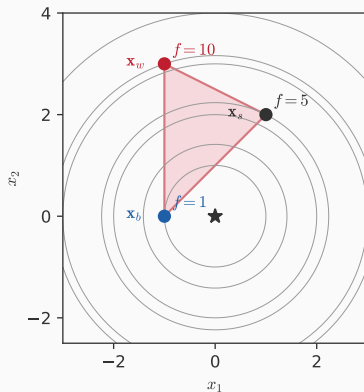
The grid (3.2.2) finds promising regions, but keeps spending evaluations where we already know it does not matter: as in random search, the points **do not use what has already happened**.

In 2D, with $f(x_1, x_2) = x_1^2 + x_2^2$, we evaluated three points:

$$f(-1, 0) = 1, \quad f(1, 2) = 5, \quad f(-1, 3) = 10.$$

We sort: $f(x_b) < f(x_s) < f(x_w)$ — the **best**, the **second worst** and the **worst**.

If we only knew these three values, where would we try next?



Three points in 2D: reflection is born

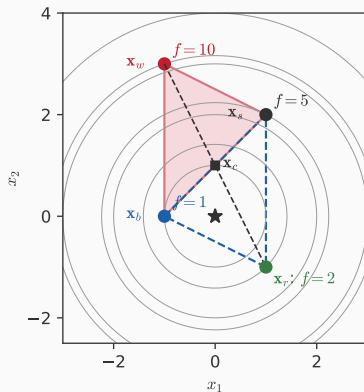
The natural answer: on the side opposite the worst point.

Centre of the other two: $x_c = \frac{x_b + x_s}{2} = (0, 1)$.

Reflect the worst through x_c :

$$x_r = x_c + (x_c - x_w) = (1, -1), \quad f(x_r) = 2.$$

The worst ($f = 10$) is replaced by a point with $f = 2$:
the triangle **flips** away from the worst vertex.



Three points in 2D: reflection is born

The natural answer: **on the side opposite the worst point.**

Centre of the other two: $x_c = \frac{x_b + x_s}{2} = (0, 1)$.

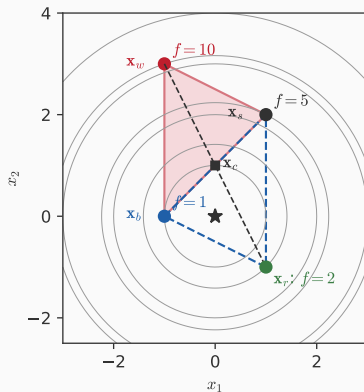
Reflect the worst through x_c :

$$x_r = x_c + (x_c - x_w) = (1, -1), \quad f(x_r) = 2.$$

The worst ($f = 10$) is replaced by a point with $f = 2$:
the triangle **flips** away from the worst vertex.

The central idea of Nelder–Mead

With a few values of f , the **geometry** of the points already evaluated tells us where to try next. No gradient was computed.



The simplex in n dimensions

A **simplex** in $\mathbb{R}^n$ is the polytope with $n + 1$ vertices:

$n = 1$: segment; $n = 2$: triangle; $n = 3$: tetrahedron.

Nondegenerate = with nonzero (n -dimensional) volume (the vertices do not all lie in a hyperplane).

Nelder–Mead keeps $n + 1$ points and uses **only their values of f** . At each iteration it:

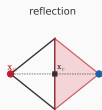
1. sorts the vertices: best x_b , second worst x_s , worst x_w ;
2. computes the centroid of the n vertices that are *not* the worst: $x_c = \frac{1}{n} \sum_{i \neq w} x_i$;
3. tries points on the line $x_w \rightarrow x_c$, on the other side.

Why it works

With $n + 1$ values of f the simplex “sees” the local slope without gradients: it moves away from the worst and heads in the direction in which f seems to decrease.

In 2D, $n = 2$: the centroid of the two best is the midpoint, as on the previous page.

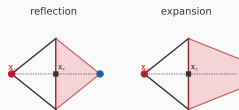
And if the reflection is very good, or bad?



All these points lie on the line defined by x_w and x_c .

- **Reflection**, $x_r = x_c + \alpha(x_c - x_w)$: if x_w is the worst, we try the other side.

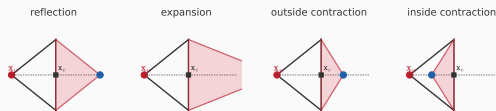
And if the reflection is very good, or bad?



All these points lie on the line defined by x_w and x_c .

- **Reflection**, $x_r = x_c + \alpha(x_c - x_w)$: if x_w is the worst, we try the other side.
- **Very good** ($f(x_r) < f(x_b)$)? It may be worth going further: **expansion**, $x_e = x_c + \gamma(x_r - x_c)$.

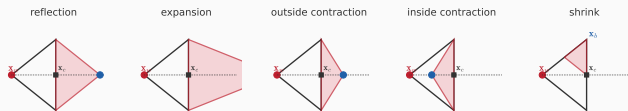
And if the reflection is very good, or bad?



All these points lie on the line defined by x_w and x_c .

- **Reflection**, $x_r = x_c + \alpha(x_c - x_w)$: if x_w is the worst, we try the other side.
- **Very good** ($f(x_r) < f(x_b)$)? It may be worth going further: **expansion**, $x_e = x_c + \gamma(x_r - x_c)$.
- **Not good enough** ($f(x_r) \geq f(x_s)$)? Perhaps we went too far: outside **contraction**, between x_c and x_r :
 $x_{oc} = x_c + \rho(x_r - x_c)$; or inside, between x_c and x_w :
 $x_{ic} = x_c - \rho(x_c - x_w)$.

And if the reflection is very good, or bad?



All these points lie on the line defined by x_w and x_c .

- **Reflection**, $x_r = x_c + \alpha(x_c - x_w)$: if x_w is the worst, we try the other side.
- **Very good** ($f(x_r) < f(x_b)$)? It may be worth going further: **expansion**, $x_e = x_c + \gamma(x_r - x_c)$.
- **Not good enough** ($f(x_r) \geq f(x_s)$)? Perhaps we went too far: outside **contraction**, between x_c and x_r : $x_{oc} = x_c + \rho(x_r - x_c)$; or inside, between x_c and x_w : $x_{ic} = x_c - \rho(x_c - x_w)$.
- **Not even contraction works?** The simplex is too large or badly placed: **shrink** towards x_b , $x_i \leftarrow x_b + \sigma(x_i - x_b)$, $\sigma = \frac{1}{2}$.

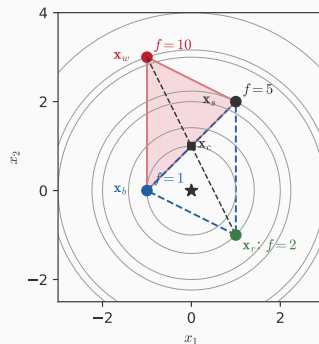
Operation	When
Expansion	$f(x_r) < f(x_b)$
Reflection accepted	$f(x_b) \leq f(x_r) < f(x_s)$
Outside contraction	$f(x_s) \leq f(x_r) < f(x_w)$
Inside contraction	$f(x_r) \geq f(x_w)$

$(\alpha, \gamma, \rho, \sigma) = (1, 2, \frac{1}{2}, \frac{1}{2})$: original coefficients (Nelder and Mead, 1965). Here α and γ are simplex coefficients, not the step α_k of 3.1.

One iteration by hand

$f(x_1, x_2) = x_1^2 + x_2^2$; the triangle from the start.

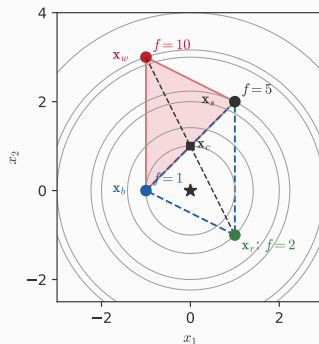
1. **Sort:** $x_b = (-1, 0)$, $f = 1$; $x_s = (1, 2)$, $f = 5$;
 $x_w = (-1, 3)$, $f = 10$.



One iteration by hand

$f(x_1, x_2) = x_1^2 + x_2^2$; the triangle from the start.

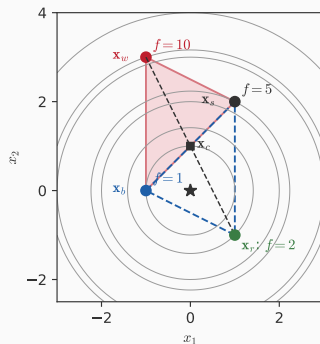
1. **Sort:** $x_b = (-1, 0)$, $f = 1$; $x_s = (1, 2)$, $f = 5$;
 $x_w = (-1, 3)$, $f = 10$.
2. **Centroid** without the worst:
 $x_c = \frac{1}{2}[(-1, 0) + (1, 2)] = (0, 1)$.



One iteration by hand

$f(x_1, x_2) = x_1^2 + x_2^2$; the triangle from the start.

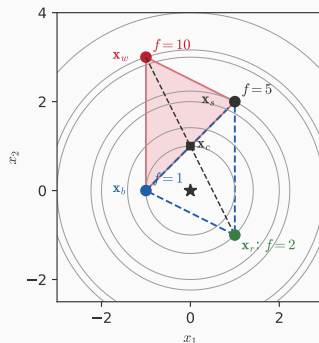
1. **Sort:** $x_b = (-1, 0)$, $f = 1$; $x_s = (1, 2)$, $f = 5$;
 $x_w = (-1, 3)$, $f = 10$.
2. **Centroid** without the worst:
 $x_c = \frac{1}{2}[(-1, 0) + (1, 2)] = (0, 1)$.
3. **Reflect:** $x_r = 2x_c - x_w = (0, 2) - (-1, 3) = (1, -1)$;
 $f(x_r) = 2$.



One iteration by hand

$f(x_1, x_2) = x_1^2 + x_2^2$; the triangle from the start.

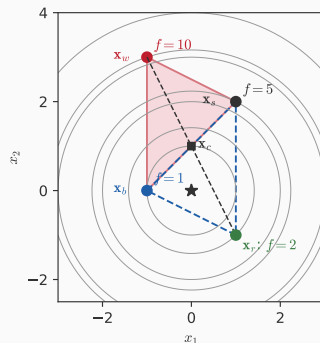
1. **Sort:** $x_b = (-1, 0)$, $f = 1$; $x_s = (1, 2)$, $f = 5$;
 $x_w = (-1, 3)$, $f = 10$.
2. **Centroid** without the worst:
 $x_c = \frac{1}{2}[(-1, 0) + (1, 2)] = (0, 1)$.
3. **Reflect:** $x_r = 2x_c - x_w = (0, 2) - (-1, 3) = (1, -1)$;
 $f(x_r) = 2$.
4. **Decide:** $f(x_b) = 1 \leq f(x_r) = 2 < f(x_s) = 5 \Rightarrow$ reflection **accepted** (it is not better than the best, so no expansion is tried; it is better than the second worst, so no contraction).



One iteration by hand

$f(x_1, x_2) = x_1^2 + x_2^2$; the triangle from the start.

1. **Sort:** $x_b = (-1, 0)$, $f = 1$; $x_s = (1, 2)$, $f = 5$;
 $x_w = (-1, 3)$, $f = 10$.
2. **Centroid** without the worst:
 $x_c = \frac{1}{2}[(-1, 0) + (1, 2)] = (0, 1)$.
3. **Reflect:** $x_r = 2x_c - x_w = (0, 2) - (-1, 3) = (1, -1)$;
 $f(x_r) = 2$.
4. **Decide:** $f(x_b) = 1 \leq f(x_r) = 2 < f(x_s) = 5 \Rightarrow$ reflection **accepted** (it is not better than the best, so no expansion is tried; it is better than the second worst, so no contraction).
5. **New simplex:** $\{(-1, 0), (1, -1), (1, 2)\}$, with $f = 1, 2, 5$. It cost **one** evaluation of f . At the next iteration, the worst is $(1, 2)$.



The new triangle already contains the minimum $(0, 0)$; the following iterations will shrink it around the minimum.

The algorithm

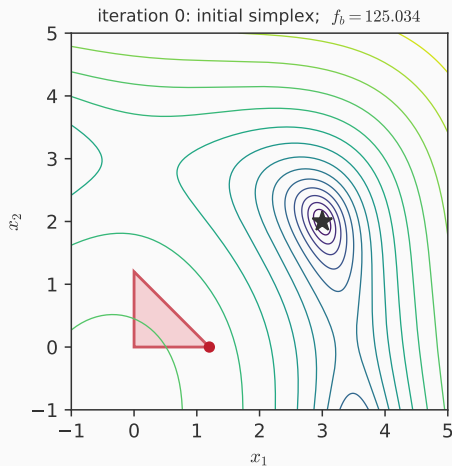
Nelder–Mead ($\alpha = 1$, $\gamma = 2$, $\rho = \frac{1}{2}$, $\sigma = \frac{1}{2}$)

```
1: choose  $n + 1$  vertices; evaluate  $f$ 
2: loop
3:   sort:  $f(x_b) \leq \dots \leq f(x_s) \leq f(x_w)$ 
4:   stop if  $f(x_w) - f(x_b) \leq \varepsilon_f$  and  $\max_i \|x_i - x_b\| \leq \varepsilon_x$ 
5:    $x_c \leftarrow \frac{1}{n} \sum_{i \neq w} x_i$ ;  $x_r \leftarrow x_c + \alpha(x_c - x_w)$ 
6:   if  $f(x_r) < f(x_b)$  then ▷ expand
7:      $x_e \leftarrow x_c + \gamma(x_r - x_c)$ ;  $x_w \leftarrow x_e$  if  $f(x_e) < f(x_r)$ , else  $x_r$ 
8:   else if  $f(x_r) < f(x_s)$  then  $x_w \leftarrow x_r$  ▷ accept
9:   else if  $f(x_r) < f(x_w)$  then ▷ outside contraction
10:     $x_{oc} \leftarrow x_c + \rho(x_r - x_c)$ 
11:    if  $f(x_{oc}) \leq f(x_r)$  then  $x_w \leftarrow x_{oc}$ 
12:    else shrink
13:    end if
14:   else ▷ inside contraction
15:     $x_{ic} \leftarrow x_c - \rho(x_c - x_w)$ 
16:    if  $f(x_{ic}) < f(x_w)$  then  $x_w \leftarrow x_{ic}$ 
17:    else shrink
18:    end if
19:   end if
   shrink:  $x_i \leftarrow x_b + \sigma(x_i - x_b)$  for all  $i \neq b$ 
20: end loop
```

sort $\rightarrow$ centroid $\rightarrow$ reflect
excellent $\rightarrow$ expand; reasonable $\rightarrow$
accept;
bad $\rightarrow$ contract; still bad $\rightarrow$ shrink

- **Cost:** $n_f = 1\text{--}2$ per iteration; $n + 2$ if there is a shrink (x_r , the contraction point and n new vertices).
- **Initial simplex:** x_0 and $x_0 + h_i e_i$, $i = 1, \dots, n$, with $h_i = 0.05 \max(1, |x_{0,i}|)$: relative increment, with an absolute minimum for zero components; in the examples: edges 1.2 (Himmelblau) and 0.5 (Rosenbrock).

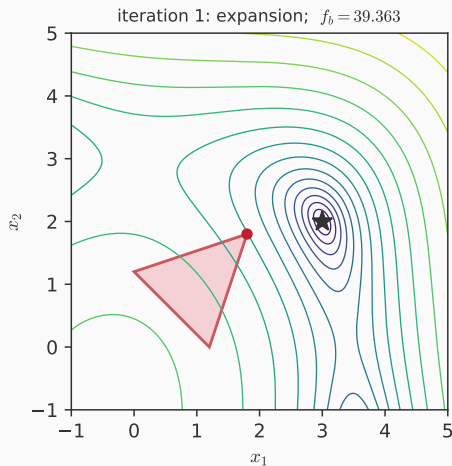
Himmelblau from (0,0): the simplex in action



Initial simplex (0,0), (1.2,0), (0,1.2).

k	operation	$\mathbf{x}_b$	$f(\mathbf{x}_b)$	$f(\mathbf{x}_w)$
0	initial	(1.200, 0.000)	125.0336	170.000

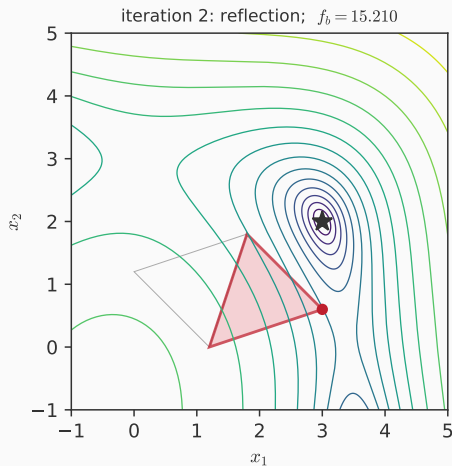
Himmelblau from (0,0): the simplex in action



Initial simplex (0,0), (1.2, 0), (0, 1.2).

k	operation	$\mathbf{x}_b$	$f(\mathbf{x}_b)$	$f(\mathbf{x}_w)$
0	initial	(1.200, 0.000)	125.0336	170.000
1	expansion	(1.800, 1.800)	39.3632	126.954

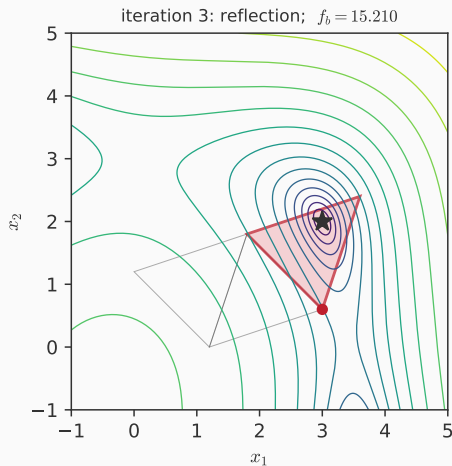
Himmelblau from (0,0): the simplex in action



Initial simplex (0, 0), (1.2, 0), (0, 1.2).

k	operation	$\mathbf{x}_b$	$f(\mathbf{x}_b)$	$f(\mathbf{x}_w)$
0	initial	(1.200, 0.000)	125.0336	170.000
1	expansion	(1.800, 1.800)	39.3632	126.954
2	reflection	(3.000, 0.600)	15.2096	125.034

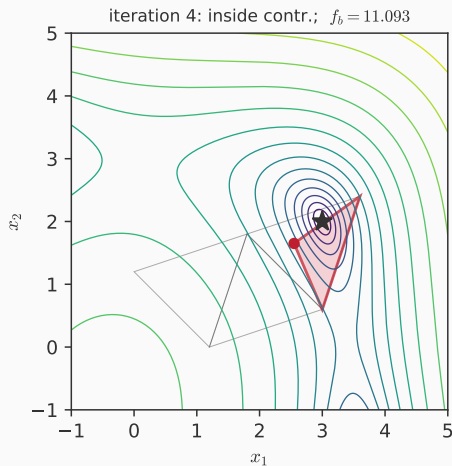
Himmelblau from (0,0): the simplex in action



Initial simplex $(0, 0)$, $(1.2, 0)$, $(0, 1.2)$.

k	operation	$\mathbf{x}_b$	$f(\mathbf{x}_b)$	$f(\mathbf{x}_w)$
0	initial	(1.200, 0.000)	125.0336	170.000
1	expansion	(1.800, 1.800)	39.3632	126.954
2	reflection	(3.000, 0.600)	15.2096	125.034
3	reflection	(3.000, 0.600)	15.2096	39.363

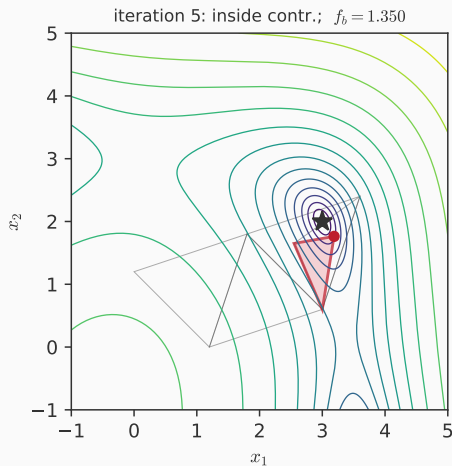
Himmelblau from (0,0): the simplex in action



Initial simplex (0,0), (1.2,0), (0,1.2).

k	operation	$\mathbf{x}_b$	$f(\mathbf{x}_b)$	$f(\mathbf{x}_w)$
0	initial	(1.200, 0.000)	125.0336	170.000
1	expansion	(1.800, 1.800)	39.3632	126.954
2	reflection	(3.000, 0.600)	15.2096	125.034
3	reflection	(3.000, 0.600)	15.2096	39.363
4	inside contr.	(2.550, 1.650)	11.0925	24.579

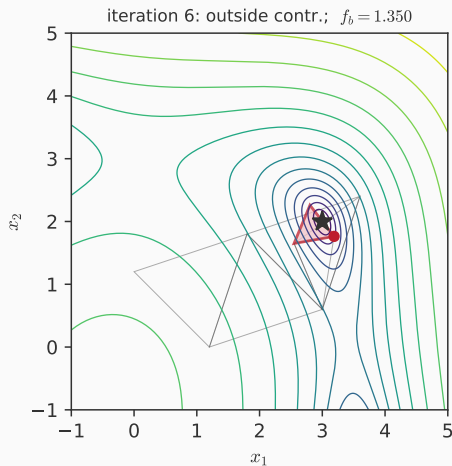
Himmelblau from (0,0): the simplex in action



Initial simplex (0,0), (1.2,0), (0,1.2).

k	operation	$\mathbf{x}_b$	$f(\mathbf{x}_b)$	$f(\mathbf{x}_w)$
0	initial	(1.200, 0.000)	125.0336	170.000
1	expansion	(1.800, 1.800)	39.3632	126.954
2	reflection	(3.000, 0.600)	15.2096	125.034
3	reflection	(3.000, 0.600)	15.2096	39.363
4	inside contr.	(2.550, 1.650)	11.0925	24.579
5	inside contr.	(3.188, 1.762)	1.3499	15.210

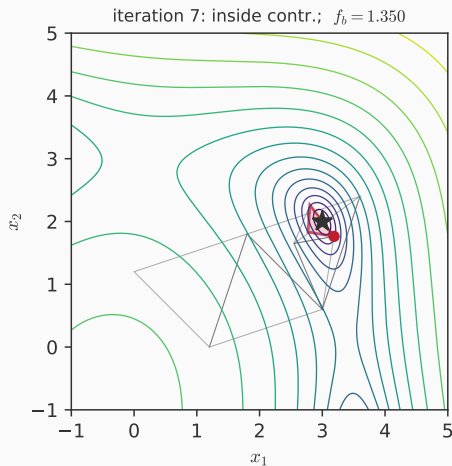
Himmelblau from (0,0): the simplex in action



Initial simplex $(0,0)$, $(1.2,0)$, $(0,1.2)$.

k	operation	$\mathbf{x}_b$	$f(\mathbf{x}_b)$	$f(\mathbf{x}_w)$
0	initial	(1.200, 0.000)	125.0336	170.000
1	expansion	(1.800, 1.800)	39.3632	126.954
2	reflection	(3.000, 0.600)	15.2096	125.034
3	reflection	(3.000, 0.600)	15.2096	39.363
4	inside contr.	(2.550, 1.650)	11.0925	24.579
5	inside contr.	(3.188, 1.762)	1.3499	15.210
6	outside contr.	(3.188, 1.762)	1.3499	11.093

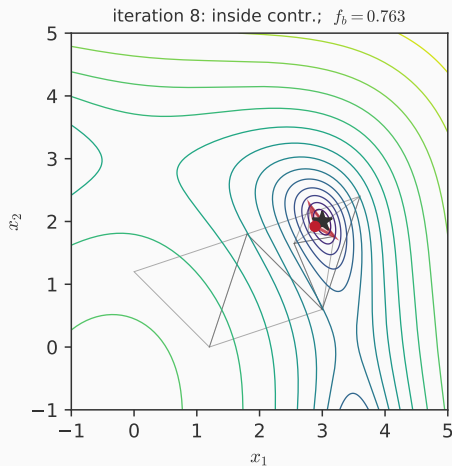
Himmelblau from (0,0): the simplex in action



Initial simplex (0,0), (1.2,0), (0,1.2).

k	operation	$\mathbf{x}_b$	$f(\mathbf{x}_b)$	$f(\mathbf{x}_w)$
0	initial	(1.200, 0.000)	125.0336	170.000
1	expansion	(1.800, 1.800)	39.3632	126.954
2	reflection	(3.000, 0.600)	15.2096	125.034
3	reflection	(3.000, 0.600)	15.2096	39.363
4	inside contr.	(2.550, 1.650)	11.0925	24.579
5	inside contr.	(3.188, 1.762)	1.3499	15.210
6	outside contr.	(3.188, 1.762)	1.3499	11.093
7	inside contr.	(3.188, 1.762)	1.3499	2.965

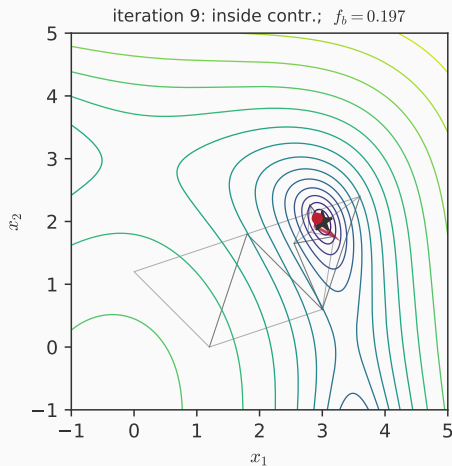
Himmelblau from (0,0): the simplex in action



Initial simplex $(0,0)$, $(1.2,0)$, $(0,1.2)$.

k	operation	$\mathbf{x}_b$	$f(\mathbf{x}_b)$	$f(\mathbf{x}_w)$
0	initial	(1.200, 0.000)	125.0336	170.000
1	expansion	(1.800, 1.800)	39.3632	126.954
2	reflection	(3.000, 0.600)	15.2096	125.034
3	reflection	(3.000, 0.600)	15.2096	39.363
4	inside contr.	(2.550, 1.650)	11.0925	24.579
5	inside contr.	(3.188, 1.762)	1.3499	15.210
6	outside contr.	(3.188, 1.762)	1.3499	11.093
7	inside contr.	(3.188, 1.762)	1.3499	2.965
8	inside contr.	(2.884, 1.921)	0.7628	1.604

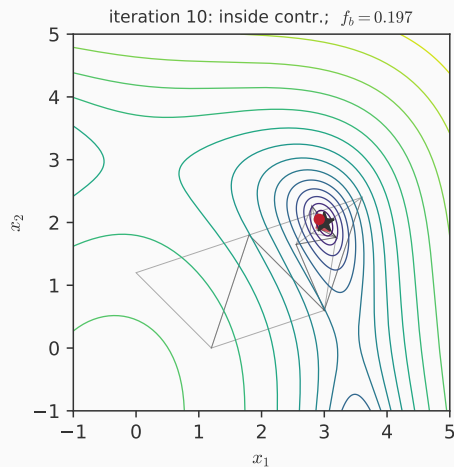
Himmelblau from (0,0): the simplex in action



Initial simplex $(0,0)$, $(1.2,0)$, $(0,1.2)$.

k	operation	$\mathbf{x}_b$	$f(\mathbf{x}_b)$	$f(\mathbf{x}_w)$
0	initial	(1.200, 0.000)	125.0336	170.000
1	expansion	(1.800, 1.800)	39.3632	126.954
2	reflection	(3.000, 0.600)	15.2096	125.034
3	reflection	(3.000, 0.600)	15.2096	39.363
4	inside contr.	(2.550, 1.650)	11.0925	24.579
5	inside contr.	(3.188, 1.762)	1.3499	15.210
6	outside contr.	(3.188, 1.762)	1.3499	11.093
7	inside contr.	(3.188, 1.762)	1.3499	2.965
8	inside contr.	(2.884, 1.921)	0.7628	1.604
9	inside contr.	(2.919, 2.050)	0.1972	1.350

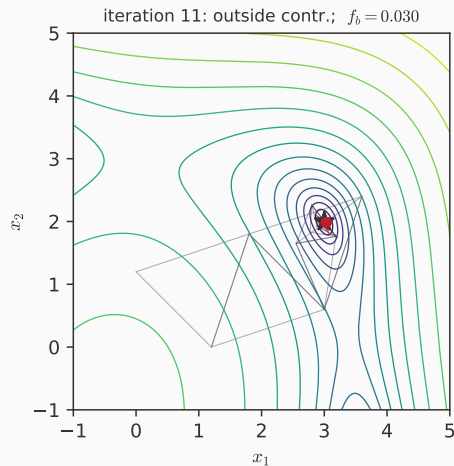
Himmelblau from (0,0): the simplex in action



Initial simplex $(0, 0)$, $(1.2, 0)$, $(0, 1.2)$.

k	operation	$\mathbf{x}_b$	$f(\mathbf{x}_b)$	$f(\mathbf{x}_w)$
0	initial	(1.200, 0.000)	125.0336	170.000
1	expansion	(1.800, 1.800)	39.3632	126.954
2	reflection	(3.000, 0.600)	15.2096	125.034
3	reflection	(3.000, 0.600)	15.2096	39.363
4	inside contr.	(2.550, 1.650)	11.0925	24.579
5	inside contr.	(3.188, 1.762)	1.3499	15.210
6	outside contr.	(3.188, 1.762)	1.3499	11.093
7	inside contr.	(3.188, 1.762)	1.3499	2.965
8	inside contr.	(2.884, 1.921)	0.7628	1.604
9	inside contr.	(2.919, 2.050)	0.1972	1.350
10	inside contr.	(2.919, 2.050)	0.1972	0.763

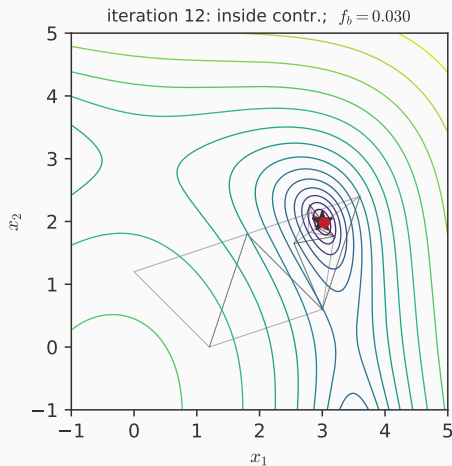
Himmelblau from (0,0): the simplex in action



Initial simplex (0,0), (1.2,0), (0,1.2).

k	operation	$\mathbf{x}_b$	$f(\mathbf{x}_b)$	$f(\mathbf{x}_w)$
0	initial	(1.200, 0.000)	125.0336	170.000
1	expansion	(1.800, 1.800)	39.3632	126.954
2	reflection	(3.000, 0.600)	15.2096	125.034
3	reflection	(3.000, 0.600)	15.2096	39.363
4	inside contr.	(2.550, 1.650)	11.0925	24.579
5	inside contr.	(3.188, 1.762)	1.3499	15.210
6	outside contr.	(3.188, 1.762)	1.3499	11.093
7	inside contr.	(3.188, 1.762)	1.3499	2.965
8	inside contr.	(2.884, 1.921)	0.7628	1.604
9	inside contr.	(2.919, 2.050)	0.1972	1.350
10	inside contr.	(2.919, 2.050)	0.1972	0.763
11	outside contr.	(3.031, 1.983)	0.0303	0.217

Himmelblau from (0,0): the simplex in action

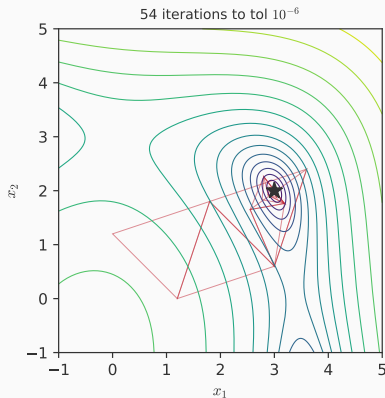


Initial simplex (0,0), (1.2,0), (0,1.2).

k	operation	$\mathbf{x}_b$	$f(\mathbf{x}_b)$	$f(\mathbf{x}_w)$
0	initial	(1.200, 0.000)	125.0336	170.000
1	expansion	(1.800, 1.800)	39.3632	126.954
2	reflection	(3.000, 0.600)	15.2096	125.034
3	reflection	(3.000, 0.600)	15.2096	39.363
4	inside contr.	(2.550, 1.650)	11.0925	24.579
5	inside contr.	(3.188, 1.762)	1.3499	15.210
6	outside contr.	(3.188, 1.762)	1.3499	11.093
7	inside contr.	(3.188, 1.762)	1.3499	2.965
8	inside contr.	(2.884, 1.921)	0.7628	1.604
9	inside contr.	(2.919, 2.050)	0.1972	1.350
10	inside contr.	(2.919, 2.050)	0.1972	0.763
11	outside contr.	(3.031, 1.983)	0.0303	0.217
12	inside contr.	(3.031, 1.983)	0.0303	0.197

The triangle **grows** when it finds descent (expansion), **flips** following the terrain (reflections) and **shrinks** near the minimum (contractions). $f(\mathbf{x}_w)$ decreases in jumps: it is the worst vertex that is replaced.

The complete path; on the computer



Converges to (3,2), $f \approx 0$: 54 iterations,
 $n_f = 106$ (tolerances 10^{-6}).

- Himmelblau has 4 minima: starting from (0,0) with the simplex to the right and upwards we reach (3,2); with the simplex (0,0), (-1.2,0), (0,-1.2) we reach (-2.81, 3.13). **Nelder-Mead is a local method**; the result depends on the initial simplex.

In MATLAB

`[x, fval] = fminsearch(f, x0)` is Nelder-Mead.
Useful options: `optimset('TolX',1e-6, 'TolFun',1e-6, 'Display','iter')`.

THE SAME IN PYTHON

```
r = scipy.optimize.minimize(f, x0,  
method='Nelder-Mead'); r.nfev counts the evaluations.
```

Implementations differ in the initial simplex and in the safeguards: same idea, different counts.

The price of Nelder–Mead

Advantages

- uses only values of f (order 0);
- simple; 1–2 evaluations per iteration ($n + 2$ with a shrink);
- adapts the position, size and shape of the simplex to the terrain;
- widely used in practice: `fminsearch`.

On Rosenbrock from $(-1.5, 2)$ it reaches $f < 10^{-4}$ with $n_f = 166$ (initial simplex with edges 0.5, the one used in the final comparison in 3.3.3); localized random search needed 975 [867, 1052] (median [Q1, Q3], 30 runs). On this problem, with these parameters.

Dimension: $n \lesssim 10$ is a common rule of thumb, not a boundary — it depends greatly on the problem.

Limitations

- local;
- depends on the initial simplex (on the same Rosenbrock, with 5% edges: $n_f = 139$);
- mostly used in low/moderate dimension: performance tends to degrade as n grows;
- it can stagnate: the simplex degenerates or shrinks away from the minimum;
- Nelder–Mead has no general convergence guarantee: there are smooth convex functions on which it stops at a nonstationary point (McKinnon, 1998);
- noise can disturb the ordering of the vertices.

Derivative-free $\neq$ robust to noise.

KEY TAKEAWAYS — 3.2.3

1. Nelder–Mead uses a simplex of $n + 1$ points and only values of f .
2. At each iteration it identifies the worst vertex and searches on the opposite side: reflection.
3. The quality of the reflection decides: expand, accept, contract or shrink.
4. The simplex adapts position, size and shape to the terrain, but the method is local.
Nelder–Mead has no general convergence guarantee: there are smooth convex functions on which it stops at a nonstationary point (McKinnon, 1998).
5. Using only values of f does not mean being robust to noise.

Next (3.2.4): the simplex changes shape according to the values of f . What if, instead, we use a fixed pattern of points around the current point — the vertices of a box — and only move or shrink it? — Box's method.

References Full list: *Course bibliography*, at the end of the course PDF.

- Nelder, J. A., Mead, R., *A simplex method for function minimization*, The Computer Journal 7(4):308–313, 1965.
- McKinnon, K. I. M., *Convergence of the Nelder–Mead simplex method to a nonstationary point*, SIAM Journal on Optimization 9(1):148–158, 1998.
- Conn, A. R., Scheinberg, K., Vicente, L. N., *Introduction to Derivative-Free Optimization*, SIAM (MPS-SIAM Series on Optimization 8), 2009 — Ch. 8.

Resources

- MATLAB: `fminsearch`, `optimset('TolX',..., 'TolFun',..., 'Display', 'iter')`.
- Python: `scipy.optimize.minimize(f, x0, method='Nelder-Mead')(r.nfev)`.